TONTADARYA COLLEGE OF ENGINEERING

(Affiliated to VTU, Belagavi & Approved by AICTE, New Delhi) Mundaragi Road, GADAG – 582101

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ANALYSIS AND DESIGN OF ALGORITHMS LABORATORY

LABORATORY COMPONENT

BCSL404

(2022 SCHEME)

FORMAT NO: QF-TCE-Lab-07

Prepared by

Ms. Sadiqa Kushtar. K. Attar M.Tech

Asst. Professor

LIST OF LABORATORY EXPERIMENTS

Sl.No	Programs No.	Problem Statement
1	Module 4 Program 1	Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.
2.	Module 4 Program 2	Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.
3.	Module 4 Program 3	a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm. b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.
4.	Module 4 Program 4	Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.
5.	Module 2 Program 1	Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.
6.	Module 4 Program 5	Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.
7.	Program 4 Module 6	Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.
8.	Module 5 Program 1	Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.
9.	Module 1 Program 1	Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.
10.	Module 2 Program 2	Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

11.	Module 2 Program 3	Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.
12.	Module 5 Program 2	Design and implement C/C++ Program for N Queen's problem using Backtracking.

Course Outcomes(Cos):

At the end of the course the student should be able to:

1. Develop programs to solve computational problems using suitable algorithm design strategy.
2. Compare algorithm design strategies by developing equivalent programs and observing running times for analysis (Empirical).
3. Make use of suitable integrated development tools to develop programs
4. Choose appropriate algorithm design techniques to develop solution to the computational and complex problems.
5. Demonstrate and present the development of program, its execution and running time(s) and record the results/inferences.

Course Articulation Matrix

Mapping of Course Outcomes (Cos) with Program Outcomes (Pos)

Course Title : Analysis and Design of Algorithms	Semester : 4- Semester
Course Code : BCSL404	Year : 2023-24

Course Outcome(Cos)/Program Outcomes(Pos)	1	2	3	4	5	6	7	8	9	10	11	12	PSO-1	PSO-2
1. Develop programs to solve computational problems using suitable algorithm design strategy.	3	2	1		2			1	2	2			3	1
2. Compare algorithm design strategies by developing equivalent programs and observing running times for analysis (Empirical).		2	1		2			1	2	2			3	1
3. Make use of suitable integrated development tools to develop programs	3	2	1		2			1	2	2			3	1
4. Choose appropriate algorithm design techniques to develop solution to the computational and complex problems.	3	2	1		2			1	2	2			3	1
5. Demonstrate and present the development of program, its execution and running time(s) and record the results/inferences.	3	2	1		2			1	2	2			3	1

PROGRAM - 1

Module 4, Program 1: Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.

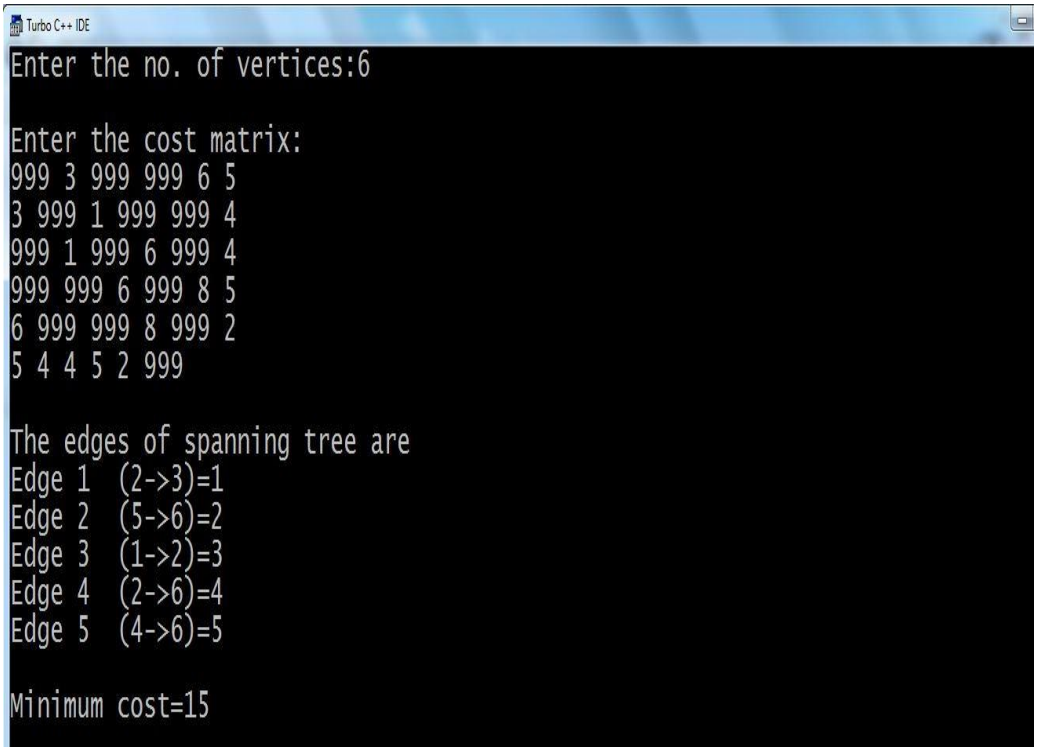
Source Code:

```
#include<stdio.h>
#include<conio.h>
int ne=1,min_cost=0;
void main()
{
int n,i,j,min,a,u,b,v,cost[20][20],parent[20];
printf("Enter the no. of vertices:");
scanf("%d",&n);
printf("\nEnter the cost matrix:\n");
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
scanf("%d",&cost[i][j]);
for(i=1;i<=n;i++)
parent[i]=0;
printf("\nThe edges of spanning tree are\n");
while(ne<n)
{
min=999;
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
if(cost[i][j]<min)
{
min=cost[i][j];
```

```
a=u=i;
b=v=j;
}
}
}
while(parent[u])
u=parent[u];
while(parent[v])
v=parent[v];
if(u!=v)
{
printf("Edge %d\t(%d->%d)=%d\n",ne++,a,b,min);
min_cost=min_cost+min;
parent[v]=u;
}
cost[a][b]=cost[a][b]=999;
}
printf("\nMinimum cost=%d\n",min_cost);

}
```

Output:


Turbo C++ IDE
Enter the no. of vertices:6
Enter the cost matrix:
999 3 999 999 6 5
3 999 1 999 999 4
999 1 999 6 999 4
999 999 6 999 8 5
6 999 999 8 999 2
5 4 4 5 2 999
The edges of spanning tree are
Edge 1 (2->3)=1
Edge 2 (5->6)=2
Edge 3 (1->2)=3
Edge 4 (2->6)=4
Edge 5 (4->6)=5
Minimum cost=15

PROGRAM - 2

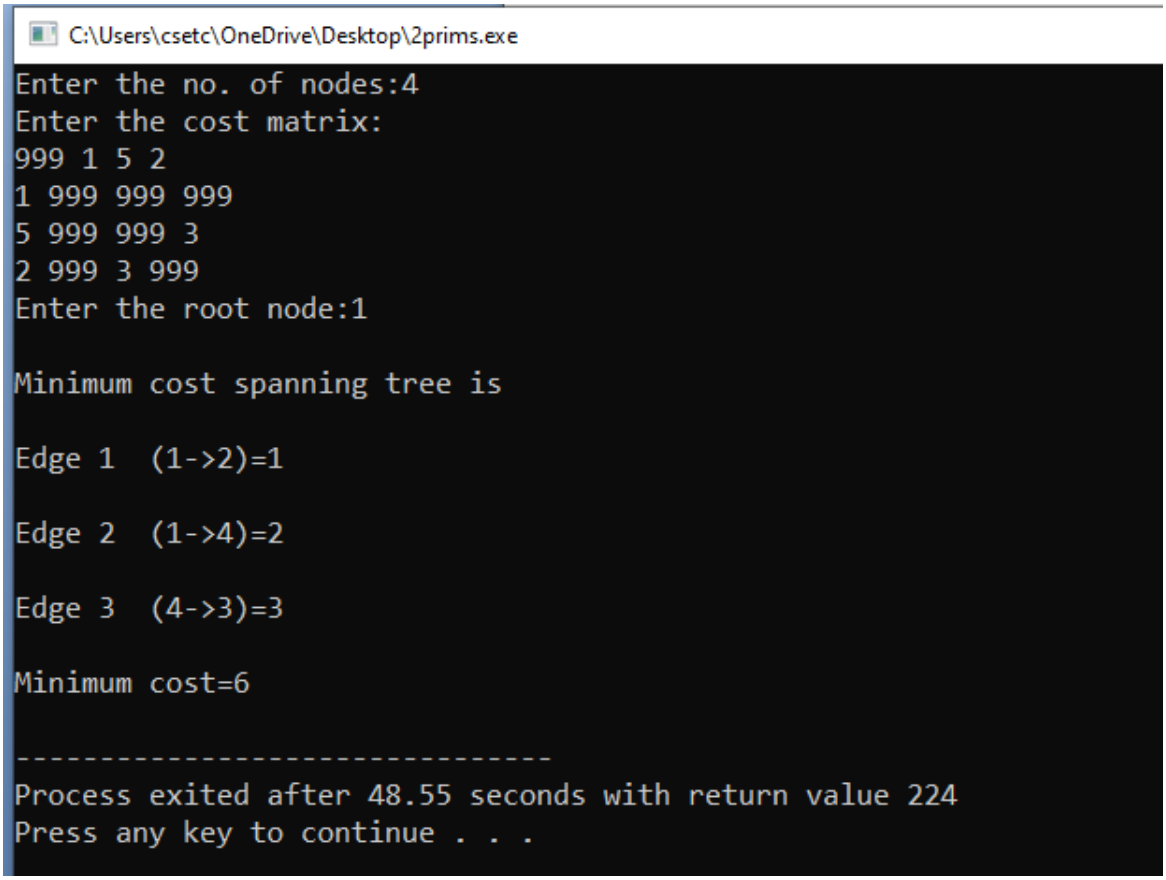
Module 4, Program 2: Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

Source Code:

```
#include<stdio.h>
int ne=1,min_cost=0;
void main()
{
int n,i,j,min,cost[20][20],a,u,b,v,source,visited[20];
printf("Enter the no. of nodes:");
scanf("%d",&n);
printf("Enter the cost matrix:\n");
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
scanf("%d",&cost[i][j]);
}
}
for(i=1;i<=n;i++)
visited[i]=0;
printf("Enter the root node:");
scanf("%d",&source);
visited[source]=1;
printf("\nMinimum cost spanning tree is\n");
while(ne<n)
{
min=999;
for(i=1;i<=n;i++)
```

```
{
for(j=1;j<=n;j++)
{
if(cost[i][j]<min)
if(visited[i]==0)
continue;
else
{
min=cost[i][j];
 a=u=i;
b=v=j;
}
}
}
if(visited[u]==0||visited[v]==0)
{
printf("\nEdge %d\t(%d->%d)=%d\n",ne++,a,b,min); min_cost=min_cost+min;
visited[b]=1;
}
cost[a][b]=cost[b][a]=999;
}
printf("\nMinimum cost=%d\n",min_cost); getch();
}
```

Output:


C:\Users\csetc\OneDrive\Desktop\2prims.exe
Enter the no. of nodes:4
Enter the cost matrix:
999 1 5 2
1 999 999 999
5 999 999 3
2 999 3 999
Enter the root node:1
Minimum cost spanning tree is
Edge 1 (1->2)=1
Edge 2 (1->4)=2
Edge 3 (4->3)=3
Minimum cost=6

Process exited after 48.55 seconds with return value 224
Press any key to continue . . .

PROGRAM - 3

Module 4, Program 3a: Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.

Source Code :

```
#include<stdio.h>
#include<conio.h>
#define INF 999
int min(int a,int b)
{
   return(a<b)?a:b;
}
void floyd(int p[][10],int n)
{
   int i,j,k;
   for(k=1; k<=n; k++)
      for(i=1; i<=n; i++)
         for(j=1; j<=n; j++)
            p[i][j]=min(p[i][j],p[i][k]+p[k][j]);
}
void main()
{
   int a[10][10],n,i,j;
   printf("\nEnter the n value:");
   scanf("%d",&n);
   printf("\nEnter the graph data:\n");
   for(i=1; i<=n; i++)
      for(j=1; j<=n; j++)
   scanf("%d",&a[i][j]);
   floyd(a,n);
```

```
    printf("\nShortest path matrix\n");
    for(i=1; i<=n; i++)
    {
        for(j=1; j<=n; j++)
            printf("%d ",a[i][j]);
        printf("\n");
    }
    getch();
}
```

Output:

```
Enter the n value:4

Enter the graph data:
0 999 3 999
2 0 999 999
999 7 0 1
6 999 999 0

Shortest path matrix
0 10 3 4
2 0 5 6
7 7 0 1
6 16 9 0
```

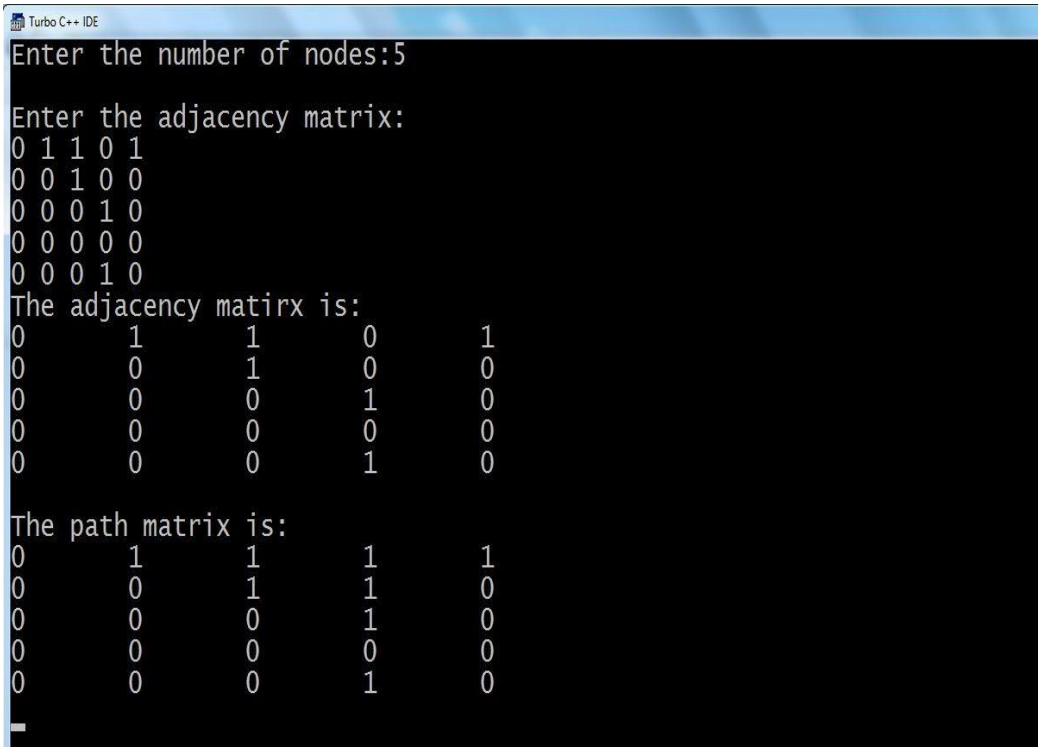
Module 4, Program 3b: Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.

Source Code:

```
#include<stdio.h>
void warshall(int[10][10],int);
void main()
{
int a[10][10],i,j,n;
printf("Enter the number of nodes:");
scanf("%d",&n);
printf("\nEnter the adjacency matrix:\n");
 for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
scanf("%d",&a[i][j]);
printf("The adjacency matirx is:\n");
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
printf("%d\t",a[i][j]);
}
printf("\n");
}
warshall(a,n);
}
void warshall(int p[10][10],int n)
{
int i,j,k;
for(k=1;k<=n;k++)
```

```
{
for(j=1;j<=n;j++)
{
for(i=1;i<=n;i++)
{
if((p[i][j]==0) || (p[i][k]==1) && (p[k][j]==1))
{
p[i][j]=1;
}
}
}
}
printf("\nThe path matrix is:\n");
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
printf("%d\t",p[i][j]);
}
printf("\n");
}
}
```

Output:


Turbo C++ IDE
Enter the number of nodes:5
Enter the adjacency matrix:
0 1 1 0 1
0 0 1 0 0
0 0 0 1 0
0 0 0 0 0
0 0 0 1 0
The adjacency matirx is:
0 1 1 0 1
0 0 1 0 0
0 0 0 1 0
0 0 0 0 0
0 0 0 1 0
The path matrix is:
0 1 1 1 1
0 0 1 1 0
0 0 0 1 0
0 0 0 0 0
0 0 0 1 0

PROGRAM - 4

Module 4, Program 4: Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.

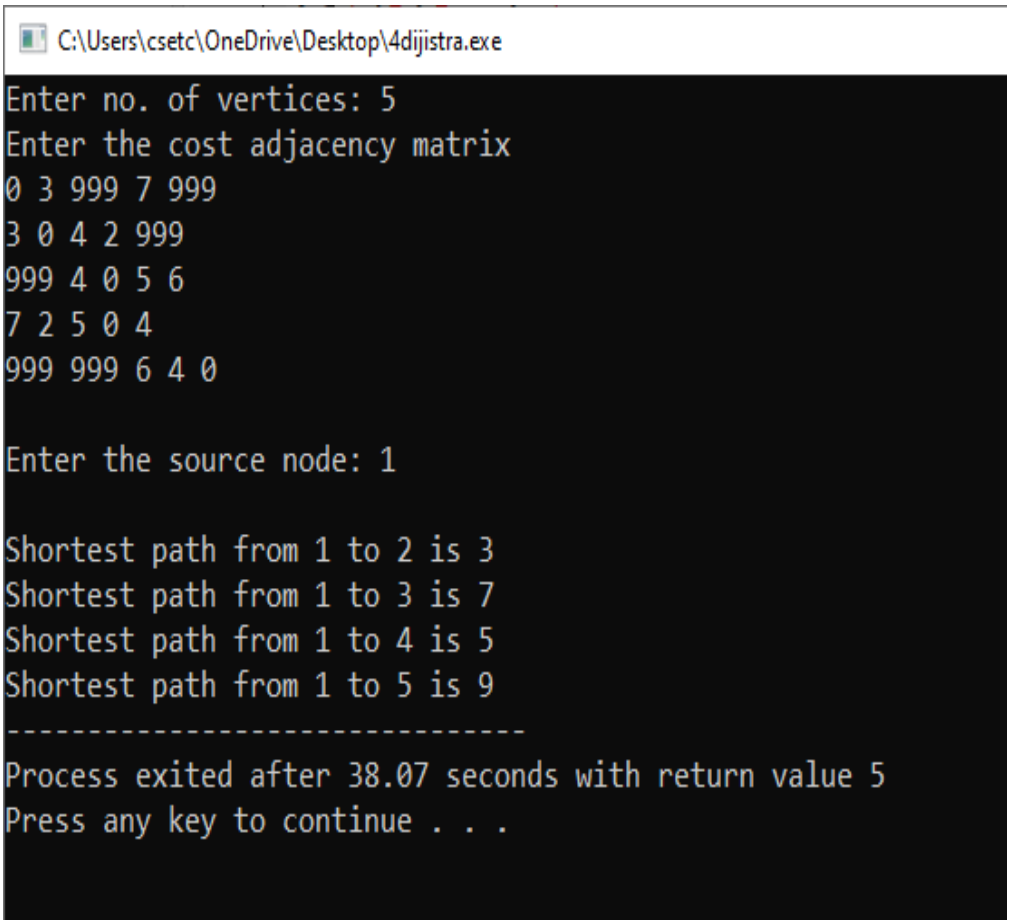
Source Code:

```
#include<stdio.h>
void dij(int,int [20][20],int [20],int [20],int);
void main()
{
int i,j,n,visited[20],source,cost[20][20],d[20];
printf("Enter no. of vertices: ");
scanf("%d",&n);
printf("Enter the cost adjacency matrix\n");
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
scanf("%d",&cost[i][j]);
}
}
printf("\nEnter the source node: ");
scanf("%d",&source);
dij(source,cost,visited,d,n);
 for(i=1;i<=n;i++)
if(i!=source)
printf("\nShortest path from %d to %d is %d",source,i,d[i]);
}
void dij(int source,int cost[20][20],int visited[20],int d[20],int n)
{
```

```
int i,j,min,u,w;
for(i=1;i<=n;i++)
{
visited[i]=0;
d[i]=cost[source][i];
}
visited[source]=1;
d[source]=0;
for(j=2;j<=n;j++)
{
min=999;
for(i=1;i<=n;i++)
{
if(!visited[i])
{
if(d[i]<min)
{
min=d[i];
 u=i;
}
}
}       //for i
visited[u]=1;
for(w=1;w<=n;w++)
{
if(cost[u][w]!=999 && visited[w]==0)
{
 if(d[w]>cost[u][w]+d[u])
d[w]=cost[u][w]+d[u];
}
```

```
} //for w
} // for j
}
```

Output:



PROGRAM – 5

Module 2, Program 1: Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

Source Code:

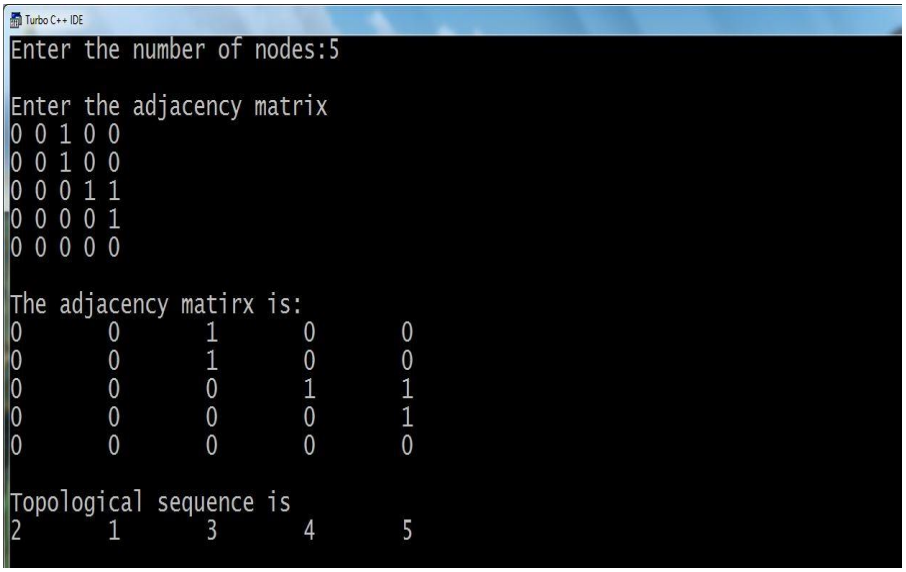
```
#include<stdio.h>

void findindegree(int [10][10],int[10],int);
void topological(int,int [10][10]);
void main()
{
int a[10][10],i,j,n;
printf("Enter the number of nodes:");
scanf("%d",&n);
printf("\nEnter the adjacency matrix\n");
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
scanf("%d",&a[i][j]);
printf("\nThe adjacency matirx is:\n");
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
printf("%d\t",a[i][j]);
}
printf("\n");
}
topological(n,a);
}
void findindegree(int a[10][10],int indegree[10],int n)
```

```
{
int i,j,sum;
for(j=1;j<=n;j++)
{
sum=0;
for(i=1;i<=n;i++)
{
sum=sum+a[i][j];
}
indegree[j]=sum;
}
}
void topological(int n,int a[10][10])
{
int k,top,t[100],i,stack[20],u,v,indegree[20];
k=1;
top=-1;
 findindegree(a,indegree,n);
for(i=1;i<=n;i++)
{
if(indegree[i]==0)
{
stack[++top]=i;
}
}
while(top!=-1)
{
u=stack[top--];
t[k++]=u;
for(v=1;v<=n;v++)
```

```
{
if(a[u][v]==1)
{
indegree[v]--;
if(indegree[v]==0)
{
stack[++top]=v;
}
}
}
}
printf("\nTopological sequence is\n");
for(i=1;i<=n;i++)
printf("%d\t",t[i]);
}
```

Output:



PROGRAM – 6

Module 4, Program 5. Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

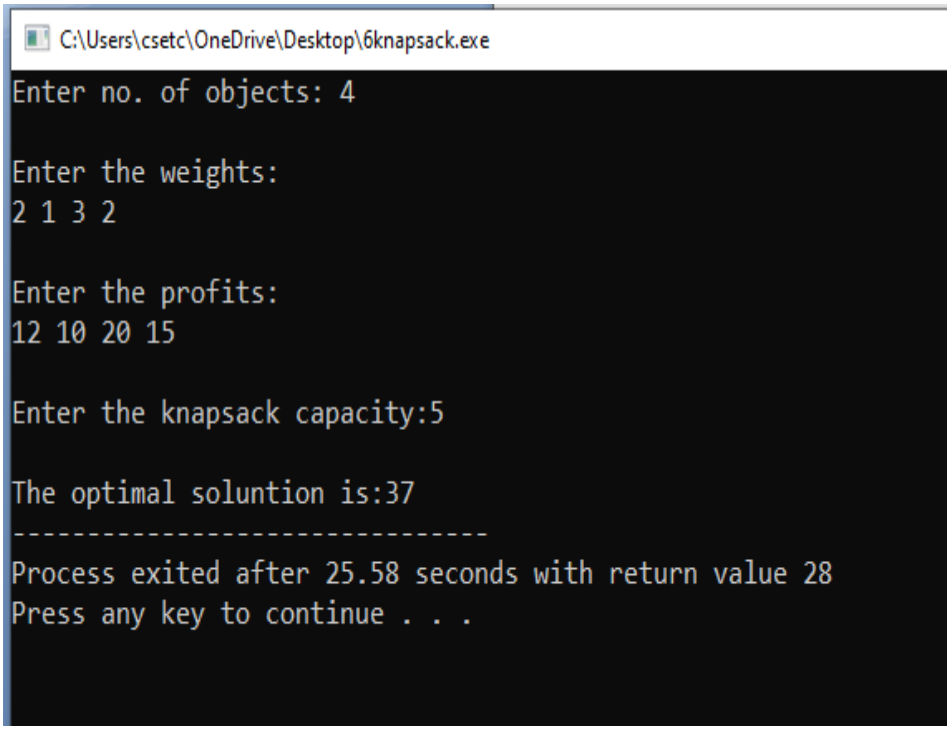
Source Code:

```
#include<stdio.h>
#define MAX 50
int p[MAX],w[MAX],n;
int knapsack(int,int);
int max(int,int);
void main()
{
int m,i,optsoln;
printf("Enter no. of objects: ");
scanf("%d",&n);
printf("\nEnter the weights:\n");
for(i=1;i<=n;i++)
scanf("%d",&w[i]);
printf("\nEnter the profits:\n");
for(i=1;i<=n;i++)
scanf("%d",&p[i]);
printf("\nEnter the knapsack capacity:");
scanf("%d",&m);
optsoln=knapsack(1,m);
printf("\nThe optimal solution is:%d",optsoln);
}
```

```
int knapsack(int i,int m)
{
if(i==n)
return (w[n]>m) ? 0 : p[n];
if(w[i]>m)
return knapsack(i+1,m);
return max(knapsack(i+1,m),knapsack(i+1,m-w[i])+p[i]);
}

int max(int a,int b)
{
 if(a>b)
return a;
else
 return b;
}
```

Output:


C:\Users\csetc\OneDrive\Desktop\6knapsack.exe
Enter no. of objects: 4
Enter the weights:
2 1 3 2
Enter the profits:
12 10 20 15
Enter the knapsack capacity:5
The optimal solution is:37
Process exited after 25.58 seconds with return value 28
Press any key to continue . . .

PROGRAM - 7

Module 4, Program 6: Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.

Source Code:

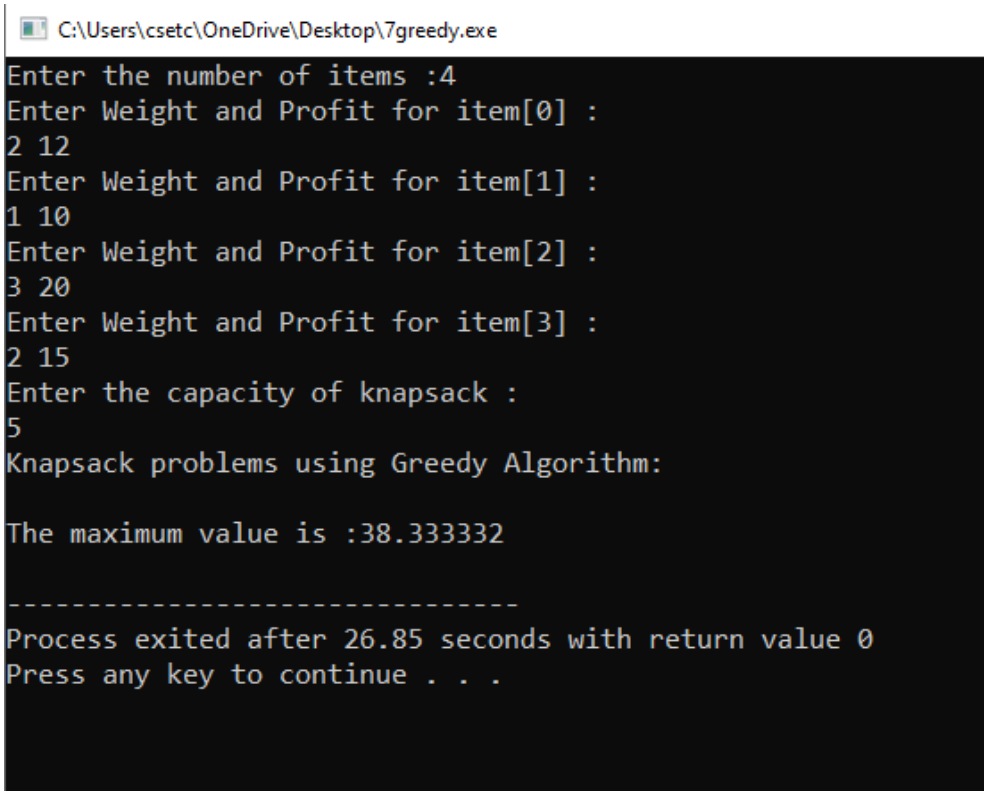
```
#include<stdio.h>
int main()
{
    float weight[50],profit[50],ratio[50],Totalvalue,temp,capacity,amount;
    int n,i,j;
    printf("Enter the number of items :");
    scanf("%d",&n);
   for (i = 0; i < n; i++)
   {
      printf("Enter Weight and Profit for item[%d] :\n",i);
      scanf("%f %f", &weight[i], &profit[i]);
   }
   printf("Enter the capacity of knapsack :\n");
   scanf("%f",&capacity);

    for(i=0;i<n;i++)
    ratio[i]=profit[i]/weight[i];

   for (i = 0; i < n; i++)
     for (j = i + 1; j < n; j++)
       if (ratio[i] < ratio[j])
       {
          temp = ratio[j];
          ratio[j] = ratio[i];
          ratio[i] = temp;
```

```
        temp = weight[j];
        weight[j] = weight[i];
        weight[i] = temp;
        temp = profit[j];
        profit[j] = profit[i];
        profit[i] = temp;
      }
    printf("Knapsack problems using Greedy Algorithm:\n");
    for (i = 0; i < n; i++)
    {
     if (weight[i] > capacity)
        break;
      else
     {
        Totalvalue = Totalvalue + profit[i];
        capacity = capacity - weight[i];
      }
    }
      if (i < n)
      Totalvalue = Totalvalue + (ratio[i]*capacity);
    printf("\nThe maximum value is :%f\n",Totalvalue);
    return 0;
}
```

Output:


C:\Users\csetc\OneDrive\Desktop\7greedy.exe
Enter the number of items :4
Enter Weight and Profit for item[0] :
2 12
Enter Weight and Profit for item[1] :
1 10
Enter Weight and Profit for item[2] :
3 20
Enter Weight and Profit for item[3] :
2 15
Enter the capacity of knapsack :
5
Knapsack problems using Greedy Algorithm:
The maximum value is :38.333332

Process exited after 26.85 seconds with return value 0
Press any key to continue . . .

PROGRAM 8

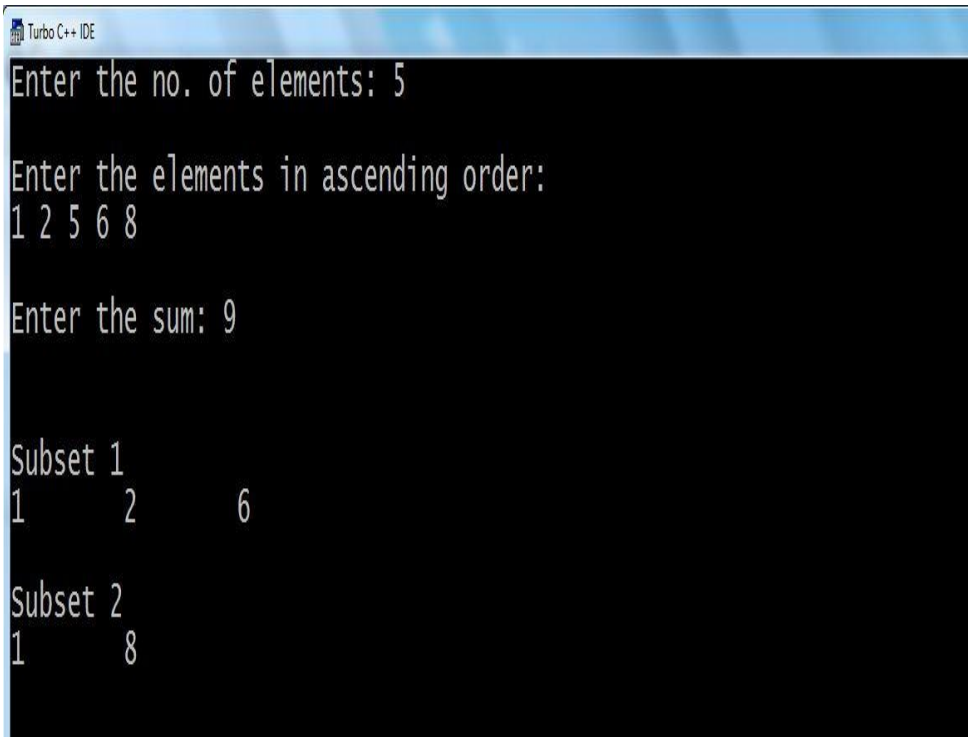
Module 5, Program 1. Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.

Source Code :

```
#include<stdio.h>
void subset(int,int,int);
int x[10],w[10],d,count=0;
void main()
{
int i,n,sum=0;
printf("Enter the no. of elements: ");
scanf("%d",&n);
printf("\nEnter the elements in ascending order:\n");
for(i=0;i<n;i++)
scanf("%d",&w[i]);
printf("\nEnter the sum: ");
scanf("%d",&d);
for(i=0;i<n;i++)
sum=sum+w[i];
if(sum<d)
{
printf("No solution\n");
return;
}
subset(0,0,sum);
if(count==0)
{
printf("No solution\n");
return;
```

```
}
}
void subset(int cs,int k,int r)
{
int i; x[k]=1;
if(cs+w[k]==d)
{
printf("\n\nSubset %d\n",++count);
for(i=0;i<=k;i++)
if(x[i]==1)
printf("%d\t",w[i]);
}
else
 if(cs+w[k]+w[k+1]<=d)
subset(cs+w[k],k+1,r-w[k]);
if(cs+r-w[k]>=d && cs+w[k]<=d)
{
x[k]=0;
subset(cs,k+1,r-w[k]);
}
}
```

Output:


Turbo C++ IDE
Enter the no. of elements: 5
Enter the elements in ascending order:
1 2 5 6 8
Enter the sum: 9
Subset 1
1 2 6
Subset 2
1 8

PROGRAM - 9

Module 1 Program 1: Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Source Code:

```
#include<stdio.h>
#include<time.h>
#define max 50000
void main()
{
int a[max],i,n;
clock_t start,end;
double time_taken;
printf("Enter the value of n:");
scanf("%d",&n);
for ( i = 0; i < n; i++)
{
   a[i] = rand() % 1000;
}
printf("\nThe array elements before\n");
for ( i = 0; i < n; i++)
{
   printf("%d\t", a[i]);
}
start = clock();
selectionsort(a, n);
end = clock();
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
printf("\nElements of the array after sorting are:\n");
for(i=0;i<n;i++)
printf("%d\t",a[i]);
printf("\nTime taken:%f", time_taken);
}
void selectionsort(int a[],int n)
{
int temp,min,i,j;
for(i=0;i<=n-2;i++)
{
        min=i;
        for(j=i+1;j<=n-1;j++)
        {
                if(a[j]<a[min])
                {
                        min=j;
                }
        }
        temp=a[i];
        a[i]=a[min];
        a[min]=temp;
}
}
```

PROGRAM - 10

Module 2, Program 2: Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Source Code:

```
#include<stdio.h>
#include<time.h>
#define max 50000
void quicksort(int a[max],int low,int high)
{
        int i, j, p, temp;
        if(low<high)
        {
                p=low;
                i=low;
                j=high;
                while(i<j)
                {
                        while(a[i]<=a[p] && i< high)
                        i++;
                        while(a[j]>a[p])
                        j--;
                        if(i<j)
                        {
                                temp=a[i];
                                a[i]=a[j];
                                a[j]=temp;
                        }
```

```
                }
                temp=a[p];
                a[p]=a[j];
                a[j]=temp;
                quicksort(a,low,j-1);
                quicksort(a,j+1,high);
        }
}
int main()
{
int x, count, a[max];
int i,n,high,low;
clock_t start,end;
double time_taken;
printf("Enter the value of n: ");
scanf("%d", &n);
for ( i = 0; i < n; i++)
{
   a[i] = rand() % 1000;
}
printf("\nThe array elements before\n");
for ( i = 0; i < n; i++)
{
   printf("%d\t", a[i]);
}
start = clock();
quicksort(a,0,n-1);
end = clock();
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nElements of the array after sorting are:\n");
```

```
for(i=0;i<n;i++)
printf("%d\t",a[i]);
printf("\nTime taken:%f", time_taken);
}
```

PROGRAM - 11

Module 2 Program 3: Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Source Code:

```
#include<stdio.h>
#include<time.h>
#define max 50000

void mergesort(int a[], int low, int high);
void merge(int a[], int low, int mid, int high);

void main()
{
        int a[max],i,n;
        clock_t start,end;
        double time_taken;

        printf("Enter the value of n:");
        scanf("%d",&n);

        for ( i = 0; i < n; i++)
        {
                a[i] = rand() % 1000;
        }

        printf("\nThe array elements before\n");
```

```
        for ( i = 0; i < n; i++)
        {
                printf("%d\t", a[i]);
        }
        start = clock();
        mergesort(a,0,n-1);
        end = clock();
        time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("\nElements of the array after sorting are:\n");
        for(i=0;i<n;i++)
        printf("%d\t",a[i]);
        printf("\nTime taken:%f", time_taken);
}
void mergesort(int a[], int low, int high)
{
        int mid;
        if(low < high)
        {
                mid = (low+high)/2;
                mergesort(a, low, mid);
                mergesort(a, mid+1, high);
                merge(a, low, mid, high);
        }
}

void merge(int a[], int low, int mid, int high)
{
        int i, j, h, k, b[100000];
        h=low; i=low; j=mid+1;
```

```
        while((h<=mid) && (j<=high))
        {
                if(a[h] < a[j])
                        b[i++] = a[h++];
                else
                        b[i++] = a[j++];
        }

        for(k=j; k<=high; k++)
        {
                b[i] = a[k];
                i = i+1;
        }

        for(k=h; k<=mid; k++)
        {
                b[i] = a[k];
                i = i+1;
        }

        for(k=low; k<= high; k++)
                a[k] = b[k];
}
```

PROGRAM - 12

Module 5, Program 2: Design and implement C/C++ Program for N Queen's problem using Backtracking.

Source Code:

```
#include<stdio.h>
void nqueens(int); int place(int[],int);
void printsolution(int,int[]);
void main()
{
int n;
printf("Enter the no.of queens: ");
scanf("%d",&n);
nqueens(n);
}
void nqueens(int n)
{
int x[10],count=0,k=1;
x[k]=0;
while(k!=0)
{
x[k]=x[k]+1;
while(x[k]<=n&&(!place(x,k)))
x[k]=x[k]+1;
if(x[k]<=n)
{
if(k==n)
{
 count++;
printf("\nSolution %d\n",count);
printsolution(n,x);
```

```
}
else
{
 k++;
x[k]=0;
 }
}
else
{
k--; //backtracking
}
 }
return;
}
int place(int x[],int k)
{
int i;
for(i=1;i<k;i++)
if(x[i]==x[k]||(abs(x[i]-x[k]))==abs(i-k)) return 0;
return 1;
}
void printsolution(int n,int x[])
{
int i,j;
char c[10][10];
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
c[i][j]='X';
}
```

```
for(i=1;i<=n;i++)
c[i][x[i]]='Q';
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
printf("%c\t",c[i][j]);
}
printf("\n");
}
}
```

Output:

